

Claude работает, *пока вы спите.*

До этого вы запускали Claude вручную. Сегодня учим его работать сам: каждое утро в 8:30 присылать сводку по портфелю в Telegram, по пятницам делать недельный обзор, мгновенно реагировать на падение цены, и вести через PostHog статистику собственных запусков.

ДЛИТЕЛЬНОСТЬ

7–9 дней · **2–4** ч/день

ЧТО ВЫ ОСВОИТЕ

cron · **schedule** · фоновые задачи · **push** · мониторинг

ГЛАВНЫЙ ИТОГ

Claude работает по расписанию без вашего участия

АРТЕФАКТ

«Финансовый дворецкий» в **Telegram** + «Дневник на автопилоте»

ОБЯЗАТЕЛЬНО • АККАУНТ

Telegram-бот через @BotFather

Что это: ваш собственный Telegram-бот, через которого Claude будет писать вам сообщения и получать ответы.

Зачем сейчас: push-канал для всех автоматизаций.

Как получить: в Telegram найдите @BotFather, нажмите `/newbot`, дайте имя и username. В ответ — `token` вида `123456:ABC-DEF...`. **Никому не показывайте:** с ним любой может писать от имени бота.

Куда положить: в `.env`: `TELEGRAM_BOT_TOKEN=<token>`. На Railway/сервере — через env vars dashboard.

Свой `chat_id`: напишите боту любое сообщение, потом откройте `https://api.telegram.org/bot<TOKEN>/getUpdates` — в JSON будет `chat.id`. Запишите в `.env` как `TELEGRAM_CHAT_ID`.

ЖЕЛАТЕЛЬНО • АККАУНТ + SKILL

PostHog + skill posthog:llma-cc-setup

Что это: PostHog — продуктовая аналитика. Skill `llma-cc-setup` учит Claude отправлять туда логи каждой своей сессии (с токенами, ошибками, моделью). На дашборде вы видите, во что вам обходится автопилот.

Зачем сейчас: когда у вас 4 cron-задачи, легко не заметить, что одна крутится впустую и тратит токены. PostHog подсветит.

Как получить: регистрация на `posthog.com` (бесплатно до 1M событий/мес). Получите project API key, положите в `.env` как `POSTHOG_API_KEY`.

Установка skill:

```
$ npx @smithery/cli@latest skill add posthog/llma-cc-setup --agent claude-code
```

Планировщик задач: cron или Task Scheduler

Что это: встроенный планировщик ОС, запускающий программы по расписанию.

Linux/macOS — `cron` через `crontab -e`. *Windows* — Task Scheduler через `taskschd.msc` или `schtasks`.

Зачем сейчас: чтобы скрипт «утренний отчёт» запускался каждый день в 8:30 без вашего участия.

Альтернатива — Railway Cron: если ваш бот живёт на Railway (этап 06), там есть встроенный cron в настройках сервиса.

Проверить: на Linux/macOS `crontab -l` покажет ваш текущий список. На Windows `schtasks /query`.

schedule + loop

Что это: два встроенных skill Claude Code. `schedule` создаёт «удалённых агентов», работающих по cron в облаке Anthropic (без вашего сервера). `loop` — запускает скрипт повторно с интервалом, не отлучаясь из текущей сессии.

Зачем сейчас: для вариантов «не хочу разбираться с cron» используем `schedule`. Для «проверить статус долгого деплоя каждые 5 минут» — `loop`.

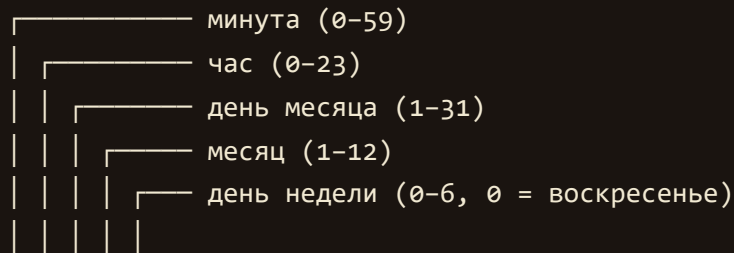
Проверить: `/skills` — должны быть в списке (часто встроены по умолчанию).

Из этапов 06–09

- Backend на Railway (этап 06) или другой сервер, где будут жить cron-задачи;
- MCP «Daily Trader» из этапа 09 (для `get_price` в скриптах);
- ANTHROPIC_API_KEY из этапа 06 (для Claude API в ваших скриптах);
- ALPHAVANTAGE_KEY из этапа 03.

Cron-расписание: как сказать машине «каждый день в 8:30»

Cron-выражение — пять полей через пробел:



минута (0-59)
час (0-23)
день месяца (1-31)
месяц (1-12)
день недели (0-6, 0 = воскресенье)

```
30 8 * * * # каждый день в 8:30  
0 19 * * 5 # каждую пятницу в 19:00  
0 9 1 * * # первое число месяца в 9:00  
*/15 * * * * # каждые 15 минут
```

Запоминать не нужно. Скажите Claude «каждый рабочий день в 8:30» — он сам построит выражение. Ваше дело — понимать, что эти пять цифр означают «когда».

Часовые пояса. Cron на сервере живёт в часовом поясе сервера. Если ваш Railway-сервер в UTC, а вам нужно «в 8:30 по Москве» — ставите `30 5 * *`. Лучшая практика — задать TZ в конфиге cron.

Где живёт расписание: cron, Railway, schedule-skill

Три варианта запустить «каждый день в 8:30»:

1. **Системный cron** на вашем компьютере. Подходит, если компьютер всегда включён. Минус — если ноутбук закрыт, расписание не сработает.
2. **Railway Cron** (или другой облачный хост). Сервис всегда поднят, расписание в настройках. Лучше всего для долгоживущих автоматизаций.

3. Skill `schedule` от Anthropic. Создаёт «remote agent», работающий в облаке Anthropic. Без вашего сервера. Идеально для учебных и личных задач.

Сравнение:

Простота		Стоимость	Когда
Системный cron		средне	бесплатно
сервер всегда включён			
Railway Cron	легко	\$5/мес	уже есть бэкенд
schedule skill	проще всего	токены Claude	лёгкие задачи без своего сервера

В курсе используем все три. «Финансовый дворецкий» с тяжёлой логикой – на Railway. Лёгкие напоминания и «дневник» – через `schedule` skill.

Push-уведомления: Telegram как универсальный канал

Когда автоматизация что-то готова сообщить, нужен канал доставки. Опции:

- Email – универсально, но часто теряется в ящике;
- Slack – хорошо для команд, требует Slack-аккаунт у всех;
- **Telegram** – мгновенно, бесплатно, у всех на телефоне.

Для курса берём Telegram. Минимум, чтобы отправить сообщение себе:

```
$ curl -X POST "https://api.telegram.org/bot$TOKEN/sendMessage" \
-d "chat_id=$CHAT_ID" \
-d "text=Hello from Claude"
```

Одна строка bash. Никаких SDK, библиотек, авторизации – просто HTTP-вызов.

В Python с библиотекой `python-telegram-bot` – ещё проще:

```
from telegram import Bot
bot = Bot(token=os.environ["TELEGRAM_BOT_TOKEN"])
bot.send_message(chat_id=os.environ["TELEGRAM_CHAT_ID"], text="...")
```

Бот может не только отправлять, но и *принимать* сообщения от вас (для «дневника на автопилоте»). Для этого – webhook на вашем backend, либо `polling` в скрипте.

Фоновые задачи и Monitor: «уйди и дай знать, когда готово»

В Claude Code есть две нативные возможности для долгих задач:

1. `run_in_background: true` – запускает Bash-команду в фоне. Главный Claude получает ID процесса и продолжает работать. Можно проверить статус позже.
2. **Monitor** – инструмент «сledi за потоком». Подходит, чтобы дождаться завершения процесса (например, build на 15 минут): Claude засыпает, просыпается на каждой строке stdout, реагирует.

Когда полезно:

- Долгий деплой – запустили в фоне, продолжаете другие задачи, проверяете через 5 минут;
- Цепочка из нескольких subagents – пока один работает, готовите brief для следующего;
- Локальный сервер – запустили `uvicorn`, переключились на тестирование через playwright.

```
> запусти uvicorn в фоне (run_in_background) на порту 8000.  
> через playwright проверь, что http://localhost:8000/docs открывается.  
> если всё ок – оставь сервер работать, мы будем тестировать.  
> если упал – покажи логи и killни процесс.
```

Идемпотентность: запустилось дважды — ничего не сломалось

Cron в реальной жизни срабатывает не всегда «один раз»: бывают дубли (сервер перезапустился, расписание сработало два раза), пропуски (сервер был выключен), сдвиги по timezone.

Правило: **каждая cron-задача должна быть идемпотентной**. Запуск дважды — не ломает ничего.

Что это значит на практике:

- Утренний отчёт может прислать дубль – неприятно, но не катастрофа.
- «Удалить старые файлы» – *не идемпотентно*: первый запуск удалил, второй удалит ещё. Решение – в скрипте проверять «есть ли что удалять».
- «Купить акцию по сигналу» – *опасно*: дубль = двойная покупка. Нужен механизм блокировки.

Дисциплина: перед каждой cron-задачей задайтесь вопросом «*что произойдёт, если она сработает дважды подряд?*». Если ответ «ничего страшного» – идём дальше. Если «деньги пропадут» – добавляем защиту.

PostHog LLM Analytics: видеть, во что обходится автопилот

Когда у вас 3–4 cron-задачи и Telegram-бот, легко не заметить, что одна из них:

- Сожгла лимит Anthropic за неделю;
- Запускается дважды (cron сошёл с ума);
- Отдаёт пустые ответы из-за истёкшего ключа.

PostHog LLM Analytics – это дашборд, на котором видно *каждый* запуск Claude из ваших скриптов: модель, токены, цена, ошибки, длительность. Skill `posthog:llm-cc-setup` добавляет в Claude Code автоматическую отправку телеметрии.

Что вы увидите на дашборде:

- Тренд токенов в день – «утренний отчёт стал жирнее, надо сократить контекст»;
- Распределение по моделям – «слишком много Orus, переключить на Sonnet»;
- Top-N ошибок – «90% сбоев из-за rate limit Alpha Vantage»;
- Cost-per-task – «каждый утренний отчёт обходится в 5 центов».

Когда вы *не следите*, автопилот тихо разоряет вас на токенах. Когда следите – видите, что оптимизировать.

Дисциплина автопилота: четыре правила

Перед тем как ставить задачу «работать самостоятельно», прогоните её по четырём правилам:

- 1. Идемпотентность.** Что произойдёт, если задача сработает дважды? Если плохо — добавьте защиту (флаг-файл, lock в БД, проверка «уже было сегодня»).
- 2. Граница времени.** Сколько максимум может длиться задача? Если больше N минут — убивать. Иначе застрявший процесс будет жечь ресурсы.
- 3. Кнопка «стоп».** Как быстро остановить автоматизацию, если что-то пошло не так? Команда `/stop_autopilot` в вашем Telegram-боте — хорошая практика.
- 4. Канал ошибок.** Куда уходят сообщения «сбой»? Если только в логи — вы их не увидите. Хороший шаблон: ошибки приходят в тот же Telegram-чат, что и успехи, но с маркером ⚠.

Без этих правил «автопилот» превращается в «чёрную дыру», в которой что-то происходит, но вы не знаете что. С ними — вы контролируете автономный процесс с любого экрана.

PRACTICUM

Прогон · сообщение в Telegram по cron

1. Создайте Telegram-бота через @BotFather, получите `TELEGRAM_BOT_TOKEN`.
2. Напишите боту любое сообщение,
в `https://api.telegram.org/bot$TOKEN/getUpdates` найдите `chat.id` — это ваш `TELEGRAM_CHAT_ID`. Положите оба в `.env`.
3. Создайте папку `autopilot-warmup`, запустите `claude`.
4. Промпт:

```
> составь bash-скрипт ping.sh:  
> читает .env, шлёт через telegram api сообщение  
> "🟢 autopilot ping at $(date '+%Y-%m-%d %H:%M')".  
> протестируй — запусти, проверь что сообщение пришло.
```

5. Если пришло — добавьте в cron на каждые 5 минут (для теста):

```
> добавь cron-задачу: запускать ping.sh каждые 5 минут.  
> на linux/macos — через crontab.  
> на windows — через schtasks.  
> покажи мне получившуюся запись.
```

6. Дождитесь следующих 5 минут — ещё одно сообщение должно прийти. Если да — cron работает.
7. Удалите тестовый cron, чтобы не спамить себя:

```
> убери задачу из cron / schtasks. подтверди что она удалилась.
```

8. Альтернативный путь — через skill `schedule`:

```
> используя skill schedule, создай remote agent
> "test-ping": каждый день в 12:00 шлёт мне в telegram
> короткое сообщение "📅 daily ping".
> покажи список моих текущих агентов.
```

9. Сравните: где удобнее? Что вы будете использовать дальше – OS-cron или skill? Что мы тренируем: **цепочку «cron → bash → telegram»** от нуля до живого пуша на ваш телефон.

Финансовый дворецкий и дневник на автопилоте

ПРОЕКТ А · ПО ИНСТРУКЦИИ

«Финансовый дворецкий» в Telegram

Цель: четыре расписания, работающих параллельно. Ежедневная сводка по портфелю, пятничный обзор, real-time алерт при падении больше 1%, месячный аудит трат. Все – в один Telegram-чат, с дисциплиной из § I.

Шаги

1. В Plan Mode (этап 04) попросите Claude составить `спес.md` для «Финансового дворецкого»: четыре расписания, источники данных (portfolio-journal CSV, Daily Trader MCP), формат каждого сообщения.
2. Создайте папку `financial-butler/` на Railway-репо. Структура:

```
financial-butler/
├── scripts/
│   ├── morning_digest.py    # 8:30 каждый день
│   ├── friday_review.py     # 19:00 пятницы
│   ├── price_alert.py       # каждые 15 минут (real-time)
│   └── monthly_audit.py     # 1-е число в 9:00
├── lib/
│   ├── telegram.py          # отправка сообщений
│   ├── posthog.py           # запись телеметрии
│   └── lock.py               # идемпотентность через файл-флаг
├── .env.example              # шаблон без значений
└── railway.json              # описание cron-задач
```

3. **morning_digest.py.** Считает текущие позиции (через Daily Trader MCP `get_price`), вчерашние изменения, новости по holdings (через WebSearch + summarisation Claude API). Шлёт markdown-сводку в Telegram. Цена – кэшируется на 15 минут.

4. **friday_review.py**. За неделю: общий рост, лучший и худший тикер, прогноз на неделю через ваш skill «инвест-аналитик». Использует Claude API с Sonnet и prompt caching (этап 06).
5. **price_alert.py**. Каждые 15 минут проверяет: упала ли цена больше 1% за сутки? Если да – шлёт алерт. **Идемпотентность**: ведёт `data/alerts-sent.json`, чтобы не слать одно и то же дважды.
6. **monthly_audit.py**. 1-го числа считает: куда ушли деньги в прошлом месяце (по дашборду из этапа 04), три самых неожиданных факта.
7. Подключите PostHog: используя skill `posthog:llma-cc-setup`, добавьте телеметрию во все четыре скрипта. Каждый запуск пишет: модель, токены, длительность, успех/ошибка.
8. Настройте Railway Cron в `railway.json`:

```
{
  "$schema": "https://railway.app/railway.schema.json",
  "deploy": {
    "cronJobs": [
      { "schedule": "30 5 * * *", "command": "python scripts/morning_digest.py" },
      { "schedule": "0 16 * * 5", "command": "python scripts/friday_review.py" },
      { "schedule": "*/15 * * * *", "command": "python scripts/price_alert.py" },
      { "schedule": "0 6 1 * *", "command": "python scripts/monthly_audit.py" }
    ]
  }
}
```

(Все времена – в UTC, привычные московские часы пересчитайте.)

9. Задеплойте на Railway. Дождитесь первого срабатывания каждой задачи (для real-time alert – 15 минут, для morning digest – следующее утро). Если что-то не пришло – смотрите Railway logs.
10. Дисциплина «кнопка стоп»: добавьте отдельную команду `/pause` в вашем Telegram-боте – она ставит флаг-файл, и все четыре скрипта при старте проверяют этот флаг. Удобно, когда вы в отпуске и не хотите пушей.
11. Verification через subagent: «запусти red-team subagent. brief: представь, что один из скриптов завис. что произойдёт за неделю? где бы ты добавил защиту?»

«Дневник на автопилоте»

Цель: каждый вечер бот задаёт вам пять вопросов, по воскресеньям – пишет аналитический отчёт о вашей неделе. Бот и спрашивает, и отчитывается: вы всё ведёте через Telegram, не открывая компьютер.

Шаги

1. Создайте репо `diary-bot/`. Используйте опыт этапа 06 (FastAPI + PostgreSQL) или просто SQLite-файл.
2. Модель данных: `entries(date, q1, q2, q3, q4, q5, raw_chat)`.
3. Расписание (cron / Railway):
 - `0 19 * * *` – 22:00 по Москве (UTC+3): бот шлёт первый вопрос;
 - `0 11 * * 0` – 14:00 воскресенье: бот шлёт еженедельный отчёт.
4. Скрипт «пять вопросов»:

```
> составь python-скрипт ask_five_questions.py:
> шлёт пять вопросов поочерёдно, каждый в новом сообщении telegram.
> пять вопросов (сначала жёстко зашитые, потом будем варьировать):
> 1) что было самым ярким сегодня?
> 2) на чём вы сегодня фокусировались?
> 3) что вас выбило из колеи?
> 4) кто вам сегодня запомнился?
> 5) одна благодарность дню.
> ждёт ответ на каждый (через polling api telegram).
> сохраняет в БД entries.
```

5. Скрипт «еженедельный отчёт»:

```
> составь weekly_report.py:
> читает entries за последнюю неделю.
> через claude api (sonnet с prompt caching):
> - находит 3 повторяющиеся темы;
> - указывает 2 «эмоциональных пика» и что их вызвало;
> - находит 2 «дыры» (молчаливые дни – почему?);
> - формулирует одну рекомендацию на следующую неделю.
> шлёт markdown-отчёт в telegram.
```

6. Сделайте `/skip` в боте – пользователь может пропустить вечерний опрос.
`/replay 2024-04-15` – повторно показать пять вопросов того дня (если вы забыли). `/last` – последний еженедельный отчёт.
7. Privacy. Записи чувствительные. Зашифруйте `raw_chat` в БД (например, через `cryptography.fernet`) и ключ держите в Railway env vars. *Если ключ потеряется – данные не восстановить; это намеренно.*
8. Подключите PostHog только для *метрик*: количество ответов, длительность, ошибки. **Никаких содержательных полей** – чтобы дневник не утекал в PostHog.
9. Verification: запустите `/replay` на вчерашних вопросах, проверьте что неделя ещё не собралась (если вы только что начали). На следующий понедельник – должен прийти первый еженедельный отчёт.

ПРОЕКТ С · САМОСТОЯТЕЛЬНО

«Daily стратег» — рынок до завтрака

Цель: вариация проекта А под ваши интересы. Не только сводку портфеля, а утренний «брифинг трейдера»: ключевые движения за ночь, события дня (макро календарь), технические сигналы по вашим тикерам, и одно «слово дня» от Claude в роли стратега. Один Telegram-пуш в 7:30, читается за 90 секунд.

Подсказки

- Структура та же, что в А — Railway cron + python-скрипт + telegram api + Daily Trader MCP. Меняется содержание брифа.
- Источники: Alpha Vantage (overnight closes), economic calendar (через WebFetch с Forex Factory или похожего), ваш Daily Trader MCP для `compute_signal`.
- Длина ограничьте: 90 секунд чтения = ~250 слов, 5–7 буллетов max. Краткость — ключ.
- Идемпотентность: записывайте дату последнего успешного пуша, не шлите дважды за утро.
- Стил: попросите Claude использовать ваш output style `cfo` (этап 08a) или сделайте новый `strategist`: сухо, факты, никаких «возможно».

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

— сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ в Telegram приходит брифинг строго в 7:30 (или ваше время) каждый рабочий день;
- ✓ длина < 250 слов, читается меньше чем за 90 секунд;
- ✓ есть пять блоков: рынок overnight, макрокалендарь дня, ваши тикеры, технические сигналы, «слово дня»;
- ✓ идемпотентность работает (тестировали запуск дважды — пришёл один пуш, второй пометил «already sent»);
- ✓ в PostHog видно стоимость и токены;
- ✓ есть команда `/pause` и `/resume`, чтобы поставить на паузу на отпуск.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Прогони неделю моих брифингов через *subagent* с ролью «строгий главный редактор финансового издания». *Brief*: какой брифинг был лучшим, какой – самым слабым? Что бы я сократил, что добавил, чтобы превратить это в ежедневный платный рассылку?»

«Личная газета»: персональный дайджест по запросу

Цель: вариация проекта B по структуре («бот спрашивает + бот отчитывается»), но для другой задачи. Бот спрашивает у вас один раз: «какие три темы вам интересны на этой неделе?» и «какие три тикера / города / спортсмена / явления вы следите?». Дальше каждое утро – персональный дайджест ровно по этим темам.

Подсказки

- Используйте опыт этапа 03 (RSS) и Claude API для саммаризации.
- Хранение «интересов» недели – в `data/interests.json`.
- Воскресенье вечером – бот спрашивает «обновить интересы?». Если ответили «нет» – используем прошлые.
- Тематика на ваш выбор: *не обязательно финансы*. Архитектура, кулинария, спорт – что хотите узнавать.
- Длина дайджеста: ровно столько, сколько вы готовы прочитать с утренним кофе.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

– сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ раз в неделю бот спрашивает про интересы, сохраняет в `data/interests.json`;
- ✓ каждое утро – персональный дайджест ровно по этим темам, не шире;
- ✓ пройдена «неделя на вылов»: вы давали разные интересы, дайджест менялся;
- ✓ тематика – *не финансы* (или другая, отличная от предыдущих проектов – чтобы не дублировать);
- ✓ идемпотентность, стоп-кнопка, PostHog – всё на месте, как в A.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Запусти *subagent* с ролью «редактор *Substack*-рассылки». *Brief*: можно ли мою «личную газету» превратить в платную для маленькой аудитории? Что бы ты добавил, что убрал? Какие три темы стоит мне попробовать?»

Cron	Системный планировщик задач на Linux/macOS. Запускает программы по расписанию. Расписание задаётся cron-выражением из 5 полей. Аналог в Windows – Task Scheduler.
Cron-выражение	Строка из 5 полей: минута, час, день месяца, месяц, день недели. 30 8 * * * – «каждый день в 8:30».
Railway Cron	Облачный cron внутри Railway. Расписания задаются в <code>railway.json</code> . Удобно, когда ваш бэкенд уже там живёт.
schedule (skill)	Встроенный skill Claude Code, создающий «remote agent» на cron-расписании в облаке Anthropic. Не требует своего сервера.
loop (skill)	Встроенный skill, повторяющий скрипт через интервал в текущей сессии. Полезно для «проверь статус деплоя каждые 5 минут».
Push-уведомление	Сообщение, инициированное автоматизацией и доставленное вам мгновенно. В курсе – через Telegram Bot API.
Telegram Bot API	HTTP-API Telegram для ботов. Главные методы: <code>sendMessage</code> , <code>getUpdates</code> , <code>setWebhook</code> . Авторизация через token от @BotFather.
Идемпотентность	Свойство задачи: повторный запуск даёт тот же результат и не вредит. Обязательное требование для cron-задач – cron бывает срабатывает дважды.
Background task	Bash-команда, запущенная в фоне (<code>run_in_background: true</code>). Главный Claude получает PID и продолжает работу. Используется для долгих процессов: серверов, билдов, deployments.
Monitor	Инструмент Claude Code «следить за потоком». Подходит для ожидания завершения долгого процесса с реакцией на каждую новую строку stdout.
PostHog LLM Analytics	Дашборд для слежки за запусками LLM: токены, модель, цена, ошибки, длительность. Skill <code>posthog:llm-a-cc-setup</code> добавляет

автоматическую отправку телеметрии из Claude Code.

Стоп-кнопка

Команда (например, /pause в Telegram-боте), останавливающая все автоматизации одним нажатием. Реализуется через флаг-файл, который скрипты проверяют на старте.

01 Что означает **cron**-выражение `30 8 * * 1-5` ?

«Каждый рабочий день (понедельник – пятница) в 8:30». Поля: минута 30, час 8, любой день месяца, любой месяц, дни недели 1–5.

02 Где лучше держать **cron**-задачи: на своём ноутбуке, на **Railway** или через **skill schedule** ? Почему?

Зависит. Ноутбук – только если он всегда включён (редко). Railway – если у вас уже там бэкенд, удобно держать всё в одном месте. **schedule** **skill** – самое простое для учебных и личных задач, не нужен свой сервер. Все три используем в курсе под разные сценарии.

03 Что такое идемпотентность и почему она обязательна для **cron**-задач?

Идемпотентность – задача можно запустить дважды, и результат тот же, ничего не ломается. Обязательна, потому что **cron** реально срабатывает дважды (перезапуск сервера, сдвиги *timezone*). «Купить акцию по сигналу» неидемпотентно: дубль = двойная покупка. Защита – флаг-файл «уже сделано».

04 Чем отличается **background task** от **cron**-задачи?

Background task – запуск программы в *фоне внутри текущей сессии*, она работает «параллельно» с Claude. Cron – запуск *по расписанию*, в будущем, без вашего участия. Background для «сейчас, но не блокирует», **cron** для «регулярно».

05 Почему обязательно подключать PostHog к автопилоту?

Чтобы видеть, во что обходится автопилот: токены, ошибки, длительность, тренды. Без мониторинга одна сломавшаяся задача может тихо жечь деньги недели, прежде чем заметите. PostHog – глаза, без которых автономный процесс «слепой».

06 Почему в «Дневнике на автопилоте» содержательные поля шифруются, а в PostHog уходят только метрики?

Содержание дневника – чувствительные личные данные. Они не должны быть доступны никому, кроме вас. Шифрование с ключом в env гарантирует, что даже если БД утечёт – данные не прочитают. В PostHog отправляем только обезличенные метрики (количество ответов, длительность, ошибки) – они полезны для мониторинга, но не утекают в стороннюю аналитику ваши мысли.

07 Какие четыре правила «дисциплины автопилота»?

1) Идемпотентность – повторный запуск не ломает. **2) Граница времени** – задача убивается, если работает дольше N минут. **3) Кнопка стоп** – одна команда останавливает всё. **4) Канал ошибок** – ошибки приходят туда же, куда успехи (например, в Telegram), с маркером $\triangle$.

Что я теперь умею

- ☐ Понимаю cron-выражения и умею описывать любое расписание словами.
 - ☐ Знаю три канала расписания: системный cron, Railway Cron, skill `schedule`.
 - ☐ Создал Telegram-бота, отправляю и принимаю сообщения через Bot API.
 - ☐ Делаю cron-задачи идемпотентными (флаг-файл, проверка «уже было»).
 - ☐ Подключил PostHog LLM Analytics и вижу токены/стоимость каждого запуска.
 - ☐ Применяю 4 правила автопилота: идемпотентность, граница времени, стоп-кнопка, канал ошибок.
 - ☐ Сделал четыре проекта: Финансовый дворецкий, Дневник на автопилоте, Daily стратег, Личная газета.
-
-

Готовы закрыть этап? Нажатие фиксирует прогресс на главной странице.

«Когда автоматизация работает — вы её не замечаете. Когда замечаете — что-то сломалось. Дисциплина мониторинга и есть искусство «не замечать» — долго, дешево, надёжно».

ВНУТРИ ЭТАПА

- § 0 · Подготовка
- § I · Темы
- § III · Проекты
- § V · Словарь
- § VI · Quiz
- § VII · Чек-лист

КУРС

- К оглавлению
- ← Этап 09
- Этап 11